PharmaChain Guard: Recipient-Accepted Pharmaceutical Provenance with On-Chain Safety Gates

Technical Report - Working Draft

Hemanth [Full Name] and [Team Members]
Arizona State University | CSE 540

Fall B 2026

Abstract

PharmaChain Guard is a proposed and source-implemented educational prototype for pharmaceutical batch provenance. It combines wallet roles, recipient-accepted handoffs, quantity accounting, recall and expiry checks, persistent condition holds, and off-chain certificate integrity. This working report documents the architecture, implemented source interfaces, trust assumptions, and a reproducible evaluation protocol. Dependency-free model and certificate checks were executed during authoring. Solidity compilation, EVM execution, React build, and testnet deployment remain unverified because required package downloads were blocked. No comparative benchmark numbers or production effectiveness claims are reported. The team must complete integration validation, measured results, and team-specific documentation before final submission.

1 Introduction and Problem Context

Pharmaceutical batches move between organizations whose records may be governed separately. A provenance system should answer who registered an item, which participant accepted responsibility, and which recorded safety conditions should stop further distribution. A timeline alone does not answer whether a handoff was acknowledged or whether a known recall blocks later activity. Our project focuses on making those distinctions explicit and testable.

WHO's 2017 analysis estimated that about one in ten medical products in low- and middle-income countries was substandard or falsified [1]. This is a historical estimate for a specified context, not a universal current statistic. It motivates traceability and quality assurance, but blockchain alone cannot establish the physical composition of a product or validate an external laboratory.

The FDA describes DSCSA as supporting interoperable electronic tracing of certain prescription drugs at the package level [2]. PharmaChain Guard is a batch-level classroom prototype, not a DSCSA compliance implementation. It omits package serialization, authorized-trading-partner validation, complete transaction-information exchange standards, and operational compliance processes. Public consumer access is a project design decision rather than a claim about what DSCSA mandates.

1.1 Objectives and Research Questions

We ask: (1) can a named-recipient acceptance workflow prevent unilateral custody reassignment? (2) can explicit safety predicates reject transactions after recorded recall, expiry, or an unresolved condition excursion? (3) what gas and transaction overhead accompanies these controls? (4) how can a public interface communicate document integrity and policy status without claiming physical authenticity?

Success means the implemented contract accepts the intended lifecycle, rejects specified invalid actions, and produces reproducible records and measurements. It does not mean that the prototype detects all counterfeit medicines, increases clinical safety in deployment, or outperforms all centralized systems.

1.2 Contribution and Related Work

Sylim et al. proposed blockchain for pharmaceutical distribution tracking [3]. Product provenance, role-based permissions, content-addressed storage, two-party acknowledgments, and temperature monitoring are established ideas. Our contribution is their focused integration into one small educational state machine and a controlled comparison with a deliberately simplified tracker.

The user supplied a descriptive PharmaChain concept, not its source implementation. The supplied baseline contract is therefore independently written and explicitly scoped. It cannot establish shortcomings of the original author's real code. Assertions that prior systems universally lack access control, off-chain storage, or public verification have been removed. MedRec's patient-record context is not needed as a direct pharmaceutical provenance baseline.

1.3 Scope

One manufacturer registers a batch, one distributor accepts it, and one pharmacy accepts and dispenses units from it. No splitting, merging, resale, returns, payments, NFTs, real patient data, real sensors, or regulatory integrations are included. This restriction makes custody and quantity invariants understandable while leaving a useful demonstration of provenance and safety policy.

2 Architecture and Trust Model

2.1 Components and Data Placement

The React interface uses ethers.js to query a read RPC and submit wallet-signed writes. Public lookup requires an RPC connection but no wallet signature. For a local development chain, an explicitly labeled account selector uses disposable unlocked RPC accounts. Remote signing uses MetaMask and checks that the wallet chain matches the deployment chain.

The Solidity contract stores batch metadata, the current custodian, a stage, remaining quantities, safety flags, certificate locator/digest, and append-only history. Certificate files stay off-chain, either as clearly labeled local synthetic fixtures or on IPFS. The browser hashes downloaded bytes and compares them with an explicit on-chain SHA-256 commitment.

Component	Responsibility
React + ethers	Display, role forms, wallet connection, read queries
Wallet / local RPC	Authorize transactions for selected account
Solidity contract	Enforce roles, custody, quantity, safety policies
Off-chain files / IPFS	Retain synthetic certificate bytes
SHA-256 check	Detect mismatch with committed exact bytes
Evaluation scripts	Execute tests, collect receipts and local timings

2.2 Roles and Governance

Manufacturer, distributor, pharmacy, and regulator roles are assigned by a separate administrator. The contract implements a small custom role mapping and checks permissions on-chain; hiding a UI button is not the security mechanism. The administrator can revoke roles and nominate a successor, who must accept the nomination. Administration is trusted onboarding, not a decentralized identity-verification system.

Revoking a role during a pending shipment prevents acceptance when either participant no longer has the expected role. Revocation can strand a batch until the administrator restores an appropriate role. There is intentionally no emergency custody override that could bypass the acceptance invariant. A production design would require explicit governance, key recovery, and dispute processes.

2.3 Lifecycle and Safety State

The custody stage is Manufactured, AtDistributor, AtPharmacy, or Dispensed. A pending request is separate from that stage. Proposing a shipment records an intended recipient without changing custody. Only acceptance advances custody. Cancellation consumes the old nonce so that it cannot authorize a newer request.

Recall and hold are orthogonal safety flags. Recall is permanent. Expiry is evaluated against chain time, not activated by an external scheduler. A reported out-of-range temperature sets a hold. A subsequent normal reading records more evidence but cannot release the hold; the regulator must supply a review digest. Such release does not bypass expiry or reverse a recall.

2.4 Adversaries and Assumptions

Considered attackers include arbitrary callers, unapproved recipients, stale-request replayers, parties attempting to dispense too many units, and users presenting modified certificate bytes. The contract is intended to reject their invalid operations. Chain consensus, account keys, the administrator's onboarding, and the regulator's decisions are assumed trustworthy within the prototype.

Unsolved threats include collusion, dishonest manufacturing records, cloned QR labels, omitted or false temperature reports, compromised privileged keys, malicious RPC providers, and document unavailability. Recording a report establishes that an account made an assertion, not that a sensor observed the physical shipment. All chain metadata is public even if a Solidity mapping is marked private [5].

3 Implementation Specification

3.1 Batch Registration and Immutable Metadata

`registerBatch` requires a manufacturer role, nonempty bounded metadata, a unique exact batch code per manufacturer, positive quantity, valid production/expiry times, valid temperature thresholds, and a nonzero certificate digest. A monotonically increasing numeric ID becomes the lookup key. The original manufacturer, code, dates, quantity, and document commitment have no edit API. Byte-level codes are case-sensitive; the interface does not claim semantic duplicate detection across different spellings.

3.2 Recipient-Accepted Handoffs

`proposeTransfer` requires current custody, the role expected at that stage, an approved next-stage recipient, an eligible batch, and a deadline in the next seven days and no later than expiry. The sender's role cannot skip the distributor. A per-batch nonce increments for each proposal.

`acceptTransfer` checks the recipient, nonce, live deadline, current safety state, and both expected stakeholder roles. It then advances custody and clears the request. A copied old nonce cannot accept a new proposal. The sender, named recipient, or regulator may cancel a pending request, including after the deadline or a safety hold. Shipment time and physical arrival are modeled acknowledgments, not independently measured facts.

3.3 Quantity Accounting

Only the current pharmacy can dispense. Let Q_0 be registered units, Q_r remaining units, and d_i accepted dispensing quantities. The target invariant is

$$Q_r + \sum_i d_i = Q_0, \qquad 0 \le Q_r \le Q_0. \tag{1}$$

Each request must satisfy $0 < d_i \le Q_r$. Depleting the batch sets Dispensed, while partial dispensing keeps AtPharmacy. The prototype does not allocate sub-batches to several pharmacies; conservation is over a single batch custodian.

3.4 Condition Reports, Reviews, and Recall

The current authorized custodian or regulator may submit a bounded temperature value and an evidence hash. Values use tenths of a degree Celsius to avoid floating-point arithmetic on-chain. A synthetic telemetry script preserves the exact committed UTF-8 payload alongside its digest. Values outside registered thresholds set a persistent hold; no averaging or clinical threshold inference occurs.

Only a regulator can release an existing nonrecalled hold using a nonzero review digest. Recall sets an irreversible flag, cancels any pending request, and retains custody history and remaining units. All of these records remain assertions within the trust model; neither a review digest nor a wallet role proves the quality of a real laboratory or inspection.

3.5 Off-Chain Integrity and Availability

Registration stores a locator and a separate SHA-256 digest. The verifier hashes raw fetched bytes or a user-selected local file. Reformatting JSON intentionally changes the digest because verification is byte-level. CID encoding may describe a DAG rather than a direct raw-file digest, so CID and SHA-256 are not treated as interchangeable.

The IPFS script pins through a local Kubo daemon; hosted pinning is an optional manual account step. IPFS availability depends on retention and pinning [4]. A fetch failure is shown as unverified. A successful match proves consistency with the committed bytes, not physical authenticity or issuer authorization. No pinning or wallet secret is embedded in frontend code.

3.6 Interface and Bounded Reads

The dashboard has registry cards, public batch verification, a custody/safety timeline, role-specific forms, certificate checks, and QR links. Registry cards are paged by twelve; history is paged by twenty, with a contract cap of one hundred entries per read. This bounds per-query work but does not bound total on-chain history growth. QR links require a reachable frontend and RPC; localhost links are meaningful only on the same device.

4 Evaluation Protocol and Current Evidence

4.1 Authoring-Time Verification Status

Executed: dependency-free certificate tests, specification-model tests, and offline preview HTTP checks as recorded in `VERIFICATION.md`. Browser interaction checks could not run because a browser executable was unavailable. Model tests are independent JavaScript checks and are not evidence of EVM correctness. A deterministic random sequence exercises quantity conservation over two hundred model dispensing traces.

Not executed: Solidity compilation, deployed contract integration tests, React production build, live IPFS pinning/retrieval, public-testnet deployment, and the comparative benchmark. Required packages were unavailable in the authoring environment and registry access returned HTTP 403. Tests and scripts are supplied so the team can execute and fix any resulting integration issues. No gas, throughput, or deployment success is invented here.

4.2 Baseline Definition and Fair Comparison

The independently implemented baseline has basic roles, registration, direct custody changes, a recall record, and quantity-limited dispensing. It deliberately lacks acknowledged receipt and enforcement of expiry, recall, and condition gates. It demonstrates a stated weaker specification, not a representative implementation of every existing tracker.

The Guard contract stores more metadata and richer history. Its total gas costs therefore combine policy and data-design costs. A strict ablation would hold storage and history constant while toggling individual features; that is not supplied and must not be inferred from the two-contract comparison.

4.3 Executable Experiments

`npm test` compiles both Solidity sources and runs local EVM negative/lifecycle tests. It covers unauthorized roles, duplicate codes, invalid handoff routes, wrong recipient/nonce, cancellation, deadlines, role revocation, partial/full dispensing, expiry, recall, condition holds, regulator release, and bounded history. Tests use an in-process Ganache network with the Shanghai EVM and solc 0.8.30, optimizer runs 200, viaIR enabled.

`npm run benchmark` collects receipt gas for registration, direct baseline transfer, Guard proposal and acceptance, condition hold/release, and recall. A full Guard handoff must sum both transactions. The script then attempts post-expiry and post-recall dispensing against both specifications and records whether an unsafe action was accepted.

The growth experiment registers one hundred synthetic batches. At ten, fifty, and one hundred batches, it records registration gas and a ten-trial median paged-history read time. This is an in-process local RPC measurement, not public-chain transactions per second or production latency. Metadata lengths, hardware, and background load influence readings.

4.4 Results Awaiting Execution

Measure	Baseline	Guard
Registration gas	Pending	Pending
Complete handoff gas	Pending	Pending (sum)
Post-recall dispense	Pending	Pending
Post-expiry dispense	Pending	Pending
Condition hold review	Not supported	Pending
100-batch read latency	Not measured	Pending

Replace this table with actual outputs from `evidence/benchmark.json` and retain raw test/build logs. Repeat the benchmark if variability is material. Report testnet observations separately with network, contract address, transaction hashes, and RPC context. The supplied testnet deploy script has not been exercised.

5 Analysis, Limitations, and Delivery

5.1 Comparison with a Centralized System

A centralized database can implement role permissions, digital signatures, accepted handoffs, safety gates, immutable-style audit logs, and certificate hashes. It may provide lower cost, easier correction, finer access controls, and simpler operational support. Blockchain is justified here when several organizations want a common, independently inspectable history without assigning sole ledger control to one trading participant.

This does not eliminate central trust. The prototype still trusts administrator onboarding, privileged regulator decisions, account keys, external records, and gateways. Its shared ledger trades operational complexity, transaction cost, and metadata exposure for independently replayable state changes and enforcement. A production decision should compare these governance needs, rather than assuming blockchain is always superior.

5.2 Scalability and Data Management

Certificate bytes remain off-chain, reducing on-chain data relative to storing full documents. However, metadata strings and every history entry still consume storage. Read pagination avoids unbounded return arrays, but total storage grows with events. Indexing event logs off-chain could improve search and reporting while preserving chain verification, but adds infrastructure and consistency concerns.

Two acknowledged transactions cost more than one unilateral update and introduce recipient response delay. RPC query timing is not transaction confirmation time; gas units are not monetary costs without network gas-price assumptions. The report should avoid universal scalability or cost savings claims. Throughput also depends on the chosen chain and network load, which the local experiments do not establish.

5.3 Privacy, Ethics, and Real-World Adoption

Public batch metadata may reveal commercial relationships, stock movements, and production volumes. Hashes do not make predictable inputs secret. All examples therefore use synthetic nonsensitive records. Sensitive certificates would require encryption, access controls, key management, and careful decisions about what to commit publicly. GDPR/HIPAA or DSCSA compliance is not claimed.

A misleading authenticity badge could harm users by implying physical verification. The UI instead reports provenance and policy status with an explicit limit: a record or matching file cannot establish a medicine's physical contents. A copied QR label still points to a valid digital record. Genuine adoption would require package identities, validated partner credentials, reliable inspections, interoperable data exchange, availability processes, and governance outside this prototype.

5.4 Future Work

Potential extensions include split/merge lineage with quantity conservation, multiple distribution branches, independently authenticated sensors, event indexing, signed issuer credentials for certificates, private document sharing, stronger administrative governance, and formal invariant verification. These should be evaluated incrementally. They are not prerequisites for the proposed course prototype and are not represented as implemented features.

5.5 Team Delivery and Reproducibility

The supplied 2026 overview lists proposal/team contract on October 25, contract draft on November 1, interim demonstration on November 15, and final deliverables on December 4. Individual project briefs retain 2025 dates; the actual Canvas due times and milestone overviews must be checked. Final requirements include code/README, a seven-to-ten-minute demonstration, and a four-to-eight-page IEEE two-column technical report.

The team must fill its names and agreed responsibilities, execute the actual chain workflow, complete measured results, create the repository, and record its own demo. This text is a working draft. The source and initial documentation were generated with AI assistance; disclose and review that assistance according to the course policy. No unsigned template establishes team agreement, and no local model preview substitutes for a blockchain demonstration.

References

[1] World Health Organization, "1 in 10 medical products in developing countries is substandard or falsified," Nov. 28, 2017. WHO news release.

[2] U.S. FDA, "Drug Supply Chain Security Act (DSCSA)." FDA overview, accessed Oct. 23, 2026.

[3] P. Sylim, F. Liu, A. Marcelo, and P. Fontelo, "Blockchain Technology for Detecting Falsified and Substandard Drugs in Distribution: Pharmaceutical Supply Chain Intervention," *JMIR Res. Protoc.*, vol. 7, no. 9, e10163, 2018, doi: 10.2196/10163.

[4] IPFS Docs, "Persistence, permanence, and pinning." `https://docs.ipfs.tech/concepts/persistence/`, accessed Oct. 23, 2026.

[5] Solidity Documentation, "Security considerations." `https://docs.soliditylang.org/en/latest/security-considerations.html`, accessed Oct. 23, 2026.